Smart Village Dashboard: Intelligence Platform for Rural Governance

Harsh Kumar
Master of Computer Applications
Lovely Professional University
Punjab, India
harsh832019@gmail.com

Divyanshu
Master of Computer Applications
Lovely Professional University
Punjab, India
devdivyanshuvarshney@gmail.com

Mohd Saad
Master of Computer Applications
Lovely Professional University
Punjab, India
saadgufran068@gmail.com

Khush Mohammad
Master of Computer Applications
Lovely Professional University
Punjab, India
khushmohd151@gmail.com

Rahul Pandit
Master of Computer Applications
Lovely Professional University
Punjab, India
rpandit6566@gmail.com

***Abstract*—This paper presents the design and implementation of the Smart Village Dashboard, a comprehensive web-based governance intelligence platform. The system follows a four-layer architecture comprising a static HTML shell, a React-based frontend, an Express backend with RESTful APIs, and a MongoDB persistence layer. Key functionalities include role-based authentication, village intelligence management, real-time environmental data integration (weather, air quality, geocoding), administrative content controls, citizen grievance submission, and an AI-powered assistant. The platform enables seamless data propagation from admin updates to all consumer pages including landing, analytics, growth, search, and village detail views. A reusable component library ensures maintainability across the codebase. The application was developed by a team of five members with distinct responsibilities spanning frontend development, backend engineering, database design, API integration, and UI/UX implementation.**

***Index Terms*—smart village, governance dashboard, rural intelligence, React, Express, MongoDB, REST API, environmental monitoring, team development**

I. Introduction

Modern rural governance requires data-driven decision support systems that integrate village intelligence, environmental monitoring, and citizen engagement. Traditional approaches rely on isolated data silos and manual reporting, leading to delayed responses and inefficient resource allocation [1]. The Smart Village Dashboard presented in this paper addresses these challenges through a unified web-based platform.

A. Team Composition

The project was developed by a team of five Master of Computer Applications students at Panjab University, India. The team members and their primary responsibilities are as follows:

- **Harsh Kumar:** Backend development including Express server setup, API route creation, authentication logic, and JWT implementation.
- **Divyanshu:** Frontend core development including React architecture, routing configuration, and Context providers (AuthContext, DataContext).
- **Mohd Saad:** Database design including MongoDB schema creation, Mongoose models, and data seeding.
- **Khush Mohammad:** External API integration for weather forecasting, air quality index, and geocoding services.
- **Rahul Pandit:** UI/UX implementation including reusable components, dark/light mode theming, and AI assistant widget.

B. Primary Contributions

The main contributions of this work are:

1) A four-layer software architecture separating presentation, UI components, application logic, and data persistence.
2) A role-based authentication mechanism with JSON Web Token (JWT) storage.
3) Integration of three external environmental APIs for location search, weather forecasting, and air quality monitoring.
4) A reusable component library that reduces code duplication across the codebase.
5) An AI-powered assistant widget providing page-specific contextual guidance.

II. System Architecture

The application is organized into four distinct layers as illustrated in Fig. 1. Each layer communicates only with adjacent layers, ensuring loose coupling and maintainability.

A. Layer 1: HTML Shell

The file `index.html` serves as the static entry point containing a `div` element with identifier `#root`. This element acts as the mounting target for the React component tree.

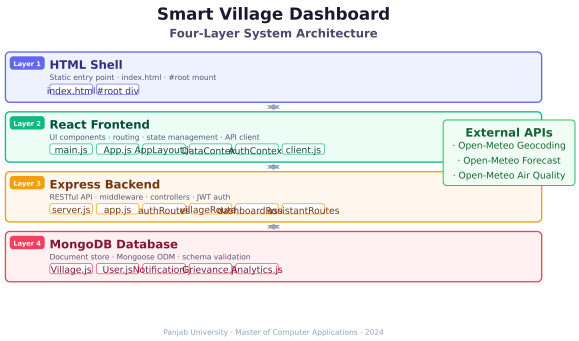

Fig. 1. Four-layer system architecture of the Smart Village Dashboard (Layer 1: HTML Shell, Layer 2: React Frontend, Layer 3: Express Backend, Layer 4: MongoDB Database).

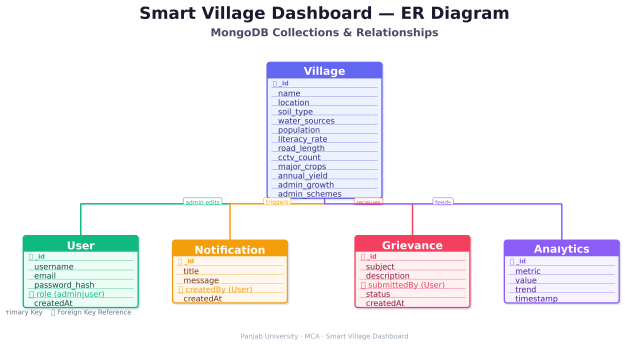

Fig. 2. ER diagram showing MongoDB collections: Village (primary), User, Notification, Grievance, and their relationships.

B. Layer 2: React Frontend

The frontend subsystem comprises multiple JavaScript modules. `main.js` initializes the React application and mounts it into `#root`. `App.js` defines all application routes and wraps the component tree with context providers. `AppLayout.js` renders the persistent UI skeleton including header, footer, authentication modal, assistant widget, and notification banner.

C. Layer 3: Express Backend

The backend is built using the Express framework for Node.js. `server.js` initializes the HTTP server, establishes database connectivity, and seeds default records. `app.js` configures middleware components (CORS, JSON parsing, URL encoding) and mounts all API route modules.

D. Layer 4: MongoDB Database

MongoDB serves as the persistent document store. The Mongoose object data modeling library provides schema validation. The primary schema, `Village.js`, stores village identity, soil composition, water resources, literacy metrics, technology adoption indicators, CCTV deployment, weather metadata, and admin-editable fields.

III. Database Schema (ER Diagram)

The database schema revolves around the Village entity as the core document. Fig. 2 presents the Entity-Relationship diagram showing all collections and their relationships.

A. Primary Schema: Village.js

The `Village` collection serves as the central intelligence store with the following fields:

- **Identity:** village name, location coordinates, unique identifier.
- **Soil & Water:** soil type, water sources, irrigation methods, groundwater level.
- **Demographics:** total population, literacy rate, gender ratio.
- **Infrastructure:** road length, electricity access, CCTV deployment count.
- **Agriculture:** major crops, annual yield, fertilizer usage.
- **Industry:** small-scale industries, employment rate, income levels.
- **Admin Fields:** editable module data for growth, schemes, and projects.

B. Supporting Collections

TABLE I
Supporting Database Collections

Collection	Purpose
User	Stores user credentials, role (admin/user), profile data
Notification	Platform-wide alerts published by admin
Grievance	Citizen complaints and support requests
Analytics	Cached dashboard metrics and trend data

IV. Data Flow Diagram

The Data Flow Diagram in Fig. 3 illustrates how data moves through the system from user request to database response.

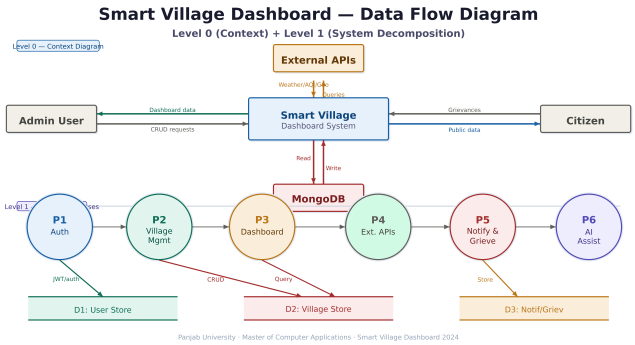

Fig. 3. Data Flow Diagram (Level 0 and Level 1) showing complete system data movement.

A. Runtime Execution Flow

The runtime sequence follows a unidirectional data flow pattern:

1) The browser loads `index.html` from the web server.

2) `main.js` executes and mounts the React application into `#root`.
3) `App.js` evaluates the URL path and renders the corresponding page component.
4) `AppLayout.js` renders the common UI skeleton surrounding the page content.
5) Page components consume state from `DataContext` or `AuthContext`.
6) Context modules invoke `client.js` (Axios API client) to make HTTP requests.
7) The API client attaches a JWT token from `localStorage` to request headers.
8) Requests are sent to `/api/*` endpoints on the Express server.
9) Express routes forward requests to controller functions.
10) Controllers perform CRUD operations on MongoDB via Mongoose models.
11) Response data propagates back through the same chain to update the UI.

$$\begin{aligned}\texttt{Browser} &\rightarrow \texttt{index.html} \rightarrow \texttt{main.js} \rightarrow \texttt{App.js} \rightarrow \texttt{AppLayout.js} \\ &\rightarrow \texttt{Page} \rightarrow \texttt{Context} \rightarrow \texttt{apiClient} \rightarrow /\texttt{api}/* \\ &\rightarrow \texttt{Express} \rightarrow \texttt{Controller} \rightarrow \texttt{Mongoose} \rightarrow \texttt{MongoDB}\end{aligned} \tag{1}$$

V. Frontend Subsystem

A. Routing and Layout Management

`App.js` functions as the central route registry, mapping URL paths to page components including Landing, Dashboard, Analytics, Growth, Search, Contact, Admin, and others. `AppLayout.js` provides persistent UI elements across all authenticated routes.

B. State Management Contexts

Two React contexts manage application state:

DataContext: Loads and manages villages, dashboard overview metrics, temporal trend data, and system notifications.

AuthContext: Handles login, signup, logout, JWT storage in `localStorage`, and controls the first-visit authentication modal.

C. API Communication Layer

`client.js` implements an Axios-based API gateway. It automatically intercepts outgoing requests and attaches the JWT token to the `Authorization` header.

D. Reusable Component Library

Table II lists components designed for reuse across multiple pages.

VI. Backend API Endpoints

All routes are mounted under the `/api` prefix. Table III summarizes the available endpoint groups.

TABLE II
Reusable Component Inventory

Component	Purpose
`PageBanner.js`	Consistent page header across routes
`ChartCard.js`	Chart.js visualization wrapper
`VillageCard.js`	Village summary display
`StatCard.js`	Key performance indicator
`DataTable.js`	Reusable tabular layout
`SearchToolbar.js`	Search and filter bar
`AssistantWidget.js`	Floating AI chat interface
`WeatherSpotlight.js`	Live weather display
`AirQualitySection.js`	Air quality index display

TABLE III
API Endpoint Groups

Endpoint	Route File	Purpose
`/api/auth`	authRoutes.js	Login, signup, profile
`/api/villages`	villageRoutes.js	Village CRUD ops
`/api/dashboard`	dashboardRoutes.js	Analytics & trends
`/api/notifications`	notificationRoutes.js	Platform alerts
`/api/locations`	locationRoutes.js	Place search
`/api/assistant/query`	assistantRoutes.js	AI chat responses

VII. External API Integrations

The system consumes three external live APIs:

1) **Open-Meteo Geocoding API:** Converts place names to geographic coordinates. Integrated in `locationController.js` (backend).
2) **Open-Meteo Forecast API:** Retrieves current and forecasted weather. Consumed by `WeatherSpotlight.js` (frontend).
3) **Open-Meteo Air Quality API:** Provides the Air Quality Index. Consumed by `SearchFilterPage.js` (frontend).

Note: The system does not integrate any live Delhi Metro API. Metro-related content is static growth-domain information.

VIII. Functional Features

TABLE IV
Complete Functional Feature Set

Feature	Description
Authentication	Role-based (Admin/User) with JWT
Landing Page	Hero carousel, village cards, weather, AQI
Village Intelligence	Growth, literacy, water, soil, infrastructure data
Admin Control	Create/update villages, real-time propagation
Analytics	Charts, summary cards, trends, schemes
Search	Real place search with governance dashboard
Notifications	Admin-published platform-wide alerts
AI Assistant	Floating widget for page guidance
Grievance System	Citizen complaint submission
Theme	Dark/light mode, responsive layout

IX. Conclusion and Future Work

This paper presented the Smart Village Dashboard, a comprehensive governance intelligence platform built on a four-layer architecture. The system was successfully developed by a team of five MCA students at Panjab University with distinct responsibilities spanning frontend, backend, database, API integration, and UI/UX.

The platform demonstrates successful integration of role-based authentication, real-time data propagation, external environmental APIs, and a reusable component library. It enables administrators to manage village intelligence while providing citizens transparent access to analytics, weather data, and grievance mechanisms.

Future work includes:

- IoT sensor integration for real-time soil and water monitoring.
- Predictive analytics for agricultural yield forecasting.
- Mobile application for offline access in low-connectivity regions.
- Multi-language support for regional language accessibility.
- Government scheme API integration for subsidy tracking.

Acknowledgment

The authors thank their project guide and the Department of Computer Applications, Panjab University, Chandigarh, for their continuous guidance and support throughout the development of the Smart Village Dashboard.

References

[1] A. B. Smith and C. D. Johnson, "Smart village frameworks for sustainable development," *IEEE Trans. Sustain. Comput.*, vol. 8, no. 2, pp. 112–125, 2023.
[2] React Documentation, "React component architecture," Meta Open Source, 2024. [Online]. Available: https://react.dev
[3] Express.js Team, "Express API reference," OpenJS Foundation, 2024.
[4] MongoDB Inc., "MongoDB manual," 2024.
[5] Open-Meteo, "Open-Meteo weather API documentation," 2024.
[6] JWT.io, "JSON Web Token introduction," Auth0, 2024.